Paper A: Deterministic Toolchain and Reproducible Artifacts for NEMA

Anonymous Author(s)

Abstract

Neural-graph programs intended for hardware require semantics that are deterministic and testable across multiple implementations and tool flows. NEMA defines a tick-based execution model for sparse neural graphs with an explicit snapshot rule, a fixed-point numeric model (RNE rounding and saturating overflow), and a bit-exact equivalence criterion based on per-tick SHA-256 digests of packed Q8.8 voltages. NEMA programs compile through a validated IR (mirrored by a proto schema) and can be exercised end-to-end via scripts that emit machine-readable reports.

We validate bit-exactness on two core benchmarks (B1 and B3) by running `nema bench verify`, which reports `ok=true` and `mismatches=[]` (mismatch=0) for both. We additionally exercise an external connectome-bundle workflow (B4) through the same manifest-driven interface. We provide hardware evidence through a self-audit gate: `audit_min -mode hardware` returns `decision=GO`, indicating toolchain detection, HLS execution and QoR capture, and Vivado implementation timing with non-null WNS recorded in `bench_report.json`. The artifact pack includes stable scripts, report paths, and a hashed manifest for reproducibility.

ACM Reference Format:
Anonymous Author(s). 2026. Paper A: Deterministic Toolchain and Reproducible Artifacts for NEMA. In *Proceedings of NEMA Paper A Draft (Paper A).* ACM, New York, NY, USA, 10 pages.

1 Introduction

NEMA targets a narrow but recurrent problem in hardware-oriented compilation: how to specify, implement, and validate neural-graph computations such that (i) executions are deterministic by construction and (ii) equivalence across implementations can be checked mechanically. In this project, the computational model is a sparse directed graph of neurons and synapses evaluated in discrete ticks. Without an explicit contract, two implementations can diverge silently due to update-order effects, floating-point drift, or "almost the same" numerics inside different toolchains. NEMA's design centers on making these failure modes observable and rejectable through a small set of gates that operate on stable, machine-readable artifacts, in line with broader observations on reproducibility variance in modern ML systems [5, 19, 24, 32].

At the semantic level, NEMA v0.1 defines a deterministic tick semantics (`nema.tick.v0.1`) with a snapshot rule that guarantees order-independent results with respect to update ordering, while still evaluating nodes in a well-defined index order. The numeric model is fixed-point: saturating overflow is the only permitted overflow behavior, and rounding is restricted to round-to-nearest ties-to-even (RNE). Nonlinearities are mediated by a fixed tanh lookup-table policy (`nema.tanh_lut.v0.1`). These choices reduce the semantic surface area of implementation-defined behavior and allow NEMA to define bit-exactness in terms of a canonical digest.

Bit-exactness is defined over per-tick state digests. For each tick, NEMA collects node voltages `V` in node-index order, packs each raw voltage as a signed int16 little-endian value (Q8.8), and computes SHA-256 over the packed byte array. Two executions are bit-exact equal when all per-tick digests match. This definition is independent of any particular simulator backend and yields a simple, stable contract that can be applied uniformly to software runners and hardware-oriented flows.

NEMA programs are represented in a validated IR (JSON shape mirrored by `nema_ir.proto`), with an explicit validator enforcing structural invariants (e.g., unique node/edge IDs, edge references to existing nodes, chemical edges directed, gap edges symmetric, non-negative conductances) and integrity constraints for external references. In particular, IR graphs may be inlined or referenced via `graph.external`; referenced bundles must exist on disk, and if a `sha256` is provided (not a placeholder token) it must match the file digest. This makes "what graph did you actually run?" part of the executable contract rather than an implicit assumption.

The repository provides an end-to-end toolchain around this contract: a textual DSL front-end that lowers deterministically to the IR, a reference simulator (`nema/sim.py`) and fixed-point library (`nema/fixed.py`), a hardware test harness (`nema/hwtest.py`), an HLS code generator (`nema/codegen/hls_gen.`) and a report schema (`tools/bench_report_schema.json`). The primary entrypoint for reproducibility is the self-audit gate `tools/audit_min.py`, which evaluates software and hardware modes and emits a JSON decision together with criteria that must hold (e.g., digest stability, bench verification success, toolchain availability, and presence of hardware report evidence).

Our minimal evaluation is defined as a set of benchmark manifests and machine-checkable metrics. The paper core includes B1 (a micro-graph sanity benchmark), B3 (a connectome-sized kernel with targetId "CE/302-7500"), and B4 (an external connectome-bundle workflow with targetId "CE/8-12"). Correctness is evaluated by `nema bench verify`, which reports `ok=true` and `mismatches=[]` (mismatch=0) for B1

Paper A, 2026,
2026.

and B3 on the traces evaluated by their manifests. Hardware evidence is evaluated through `audit_min -mode hardware`, which must return `decision=GO` and, by construction of the gate, implies toolchain detection, successful execution of the hardware pipeline (G0b), capture of QoR/report evidence (G2), and presence of Vivado implementation timing with non-null WNS (G3) in the generated `bench_report.json` artifacts. Table-style summaries of these gate outcomes are included in the artifact pack (e.g., `papers/paperA/artifacts/tables/gates_summary.*`).

Finally, NEMA's artifact story is intentionally operational: the paper's claims correspond to scripts that produce stable outputs in stable paths. The artifact pack records toolchain versions (Vivado/Vitis HLS), hashes the artifact manifest, and provides precomputed gate evidence (software and hardware audit JSON). The project's current scope explicitly excludes claims of ML accuracy advantages, biological fidelity, or on-board power/latency measurements; those are out of scope for the verified contract presented here.

1.1 Contributions

- **Deterministic semantics + numeric contract**: We specify a deterministic tick semantics (snapshot rule + index-ordered iteration) and a fixed-point numeric model (SATURATE overflow, RNE rounding), together with a bit-exact definition based on per-tick SHA-256 digests of packed Q8.8 voltages. (Artifact: `spec.md` extracts; Artifact: `nema_ir.proto`)
- **Script-verifiable bit-exactness on core benches**: We provide a manifest-driven verification command (`nema bench verify`) that reports `mismatches=[]` (mismatch=0) for B1 and B3 on the evaluated traces, making cross-implementation equivalence machine-checkable. (Artifact: `build/paper_verify/b1/*` and `build/paper_verify/b3/*` produced by the commands)
- **Hardware evidence captured and gated**: We integrate HLS execution + QoR capture (G0b/G2) and Vivado implementation timing capture with non-null WNS (G3) into the same reporting pipeline, and expose a single gate (`audit_min -mode hardware`) that must return `decision=GO`. (Table: `papers/paperA/artifacts/tables/gates_summary.md`; Artifact: `papers/paperA/artifacts/evidence/audit_hardware.json`; Artifact: `build_hw/**/bench_report.json`)
- **External bundle workflow as a first-class contract**: We support IR graphs referenced via `graph.external` with file-existence and optional SHA-256 verification, and include a benchmark (B4) that exercises this external connectome-bundle path through the same manifest/report interface. (Artifact: `benches/B4_real_connectome/manifest.json`; Artifact: `build/bench_verify_eqlozbf7/B4_celegans_external/bench_report.json`)
- **Reproducible artifact packaging with hashes and versions**: We package gate evidence and provenance metadata (git hash, toolchain versions, SHA-256 of `artifact_manifest.json`, and counts of tables/figures/evidence) to make the verification story auditable from files alone or by re-running scripts. (Artifact: `papers/paperA/artifact` + recorded SHA-256; Table: `gates_summary.*`; Fig: `papers/paperA/artifacts/figures/gates_status.txt`)

2 Semantics

2.1 Tick Semantics (`nema.tick.v0.1`)

NEMA v0.1 defines a discrete-time, deterministic operational semantics over a fixed neural graph, identified in the spec as `nema.tick.v0.1` (spec.md:141). The tick is the single state-transition step that all backends (software reference, generated C++, and hardware flows) are required to implement consistently under the fixed-point contract (§3.2) and determinism requirements (§3.4).

State. The spec fixes the observable state used for conformance to per-node voltage and activation in signed fixed-point: voltage `V` is signed `Q8.8` stored as raw `int16`, and activation `A` is signed `Q8.8` (spec.md:143–145). The tick operates over a graph-level timestep `dt` (optional; defaults to `1.0`) and a membrane time constant `tauM` that must be positive (node-level if present, else graph-level default) (spec.md:133–134).

Operational structure and snapshot rule. The tick semantics is structured around a snapshot-style update discipline (as referenced by the spec's "snapshot rule") to ensure deterministic results independent of update ordering (spec.md:169–171). Concretely, each tick is defined as a pure function from the prior tick's state snapshot to the next tick's state: all reads of per-node state that influence the tick are conceptually taken from a single consistent snapshot, and the next-state values are written only after the tick's computations are complete.

The spec explicitly requires that simulation node iteration is index-ordered (spec.md:170). Combined with the snapshot rule (spec.md:171), this yields a deterministic evaluation for each tick even in the presence of graph structures where naive in-place updates could introduce order dependence.

Normative reference. The reference implementation for the tick semantics is `nema/sim.py` (spec.md:173). The spec additionally states a maintenance constraint: if the implementation and the spec disagree, both must be updated together along with regression tests in the same change (spec.md:9). This makes the semantics executable and keeps the operational definition aligned with the conformance tests that underpin the artifact.

2.2 Fixed-Point Contract (RNE + SATURATE)

NEMA v0.1 uses a restricted, deterministic fixed-point contract. The spec states that overflow behavior is `SATURATE` only and rounding behavior is `RNE` (round-to-nearest, ties-to-even) only (spec.md:23–27). These are not configurable in v0.1 and form part of the language's semantic contract.

This explicit choice avoids floating-point execution variance that has been widely reported in both HPC and ML contexts [1, 4, 24] and is intentionally stricter than generic IEEE-754 portability expectations [12].

Raw domains. For a signed fixed-point value with total bitwidth `T`, the raw integer range is bounded by (spec.md:31–33):

- raw min: `-(2^(T-1))`
- raw max: `(2^(T-1)) - 1`

This raw-domain definition is used to define saturation and to constrain behaviors at representational limits. The contract additionally constrains several primitive operations to be deterministic in edge cases, including shifts (logical right shift is defined as a bit-pattern shift), `abs` (saturates when applied to the signed minimum), and comparisons/mux/clip (deterministic with explicit ordering) (spec.md:56–58).

Normative reference. The spec identifies `nema/fixed.py` as the reference implementation of the fixed-point behavior (spec.md:60). In v0.1, conformance to the tick semantics is defined relative to this fixed-point contract: any backend that claims to implement `nema.tick.v0.1` must implement its arithmetic such that all observable per-tick outputs (as defined by §3.3) are bit-identical under the stated rounding and saturation rules.

2.3 Bit-Exactness Definition (digests)

NEMA v0.1 defines bit-exact equivalence between executions through a per-tick digest contract. The spec states:

- "Two executions are bit-exact equal when all per-tick digests match." (spec.md:177)

The digest is computed deterministically as follows (spec.md:179–182):

1. Collect `V` in node index order.
2. Pack each raw voltage as signed int16 little-endian.
3. Compute SHA-256 over the packed byte array.

This digest definition makes the correctness criterion explicit and machine-checkable. It also makes the comparison backend-agnostic: as long as an execution can produce the same ordered sequence of raw `int16` voltages per tick, it can be validated against another execution (e.g., reference vs generated code, or software vs hardware flow) without relying on floating-point tolerances or implementation-specific traces.

Notably, the spec's digest definition is stated over `V` (spec.md:180–182). In v0.1, this makes voltage the primary observable for bit-exact equivalence, with activation `A` treated as internal to the tick computation but constrained indirectly through the same fixed-point and determinism requirements.

2.4 Determinism Threat Model and Mitigations

NEMA's determinism goal in v0.1 is narrow and explicit: eliminate nondeterminism in the semantics and in the conformance checks that validate backends against the reference.

Threat model. The primary threats addressed by the v0.1 contract are:

- Numeric nondeterminism from floating-point behavior differences across compilers, platforms, or optimization levels.
- Order-dependent updates in graph computations (e.g., in-place state updates that depend on visitation order).
- Implementation drift between the executable reference and the written specification.
- Silent input drift from external graph/bundle references changing without detection.

Mitigations (spec-driven). The spec provides concrete mitigations that are required for conformance:

1. **Fixed-point only + fixed rounding/overflow.** By mandating `SATURATE` overflow and `RNE` rounding (spec.md:23–27), and by specifying deterministic behaviors for edge-case primitives (spec.md:56–58), NEMA removes floating-point variability from the semantic core.
2. **Canonical iteration and snapshot rule.** The semantics requires index-ordered node iteration (spec.md:170) and uses a snapshot rule to guarantee order-independent results with respect to update ordering (spec.md:171). This directly targets graph-order nondeterminism.
3. **Executable reference implementation.** The tick semantics is tied to a reference implementation (`nema/sim.py`) (spec.md:173), and the spec mandates synchronized updates to implementation + spec + regression tests when inconsistencies are discovered (spec.md:9). This is a process-level mitigation against spec/implementation divergence that would otherwise undermine reproducibility.
4. **Integrity checks for external references.** For external graph references, the spec requires that referenced paths exist and that, when a `sha256` value is not a placeholder token, it must match the file digest (spec.md:136–139). This provides a deterministic input identity and prevents "same IR, different underlying bundle" situations from passing silently.

Together, these measures define a concrete determinism envelope: within the v0.1 semantics (`nema.tick.v0.1`) and fixed-point contract, executions are considered equivalent precisely when the per-tick SHA-256 digests match under the specified byte-level packing (spec.md:177–182).

3 Implementation and Artifact Reproducibility

3.1 Implementation

NEMA v0.1 is implemented as a layered toolchain that makes the semantics and the evidence machine-checkable end-to-end: a deterministic tick semantics, a fixed-point contract, an IR validator, reference execution, and optional hardware compilation stages that emit a uniform `bench_report.json` and auxiliary artifacts.

Methodologically, this positions NEMA closer to executable conformance and translation-validation style workflows [15, 20] than to fully mechanized end-to-end verified compilers, while remaining comparable to verified-HLS efforts in intent [10, 11].

DSL v0.1/v0.2 (supporting tech only). The repository contains a textual DSL front-end with a conventional compiler structure: lexer/parser, static checking, lowering, and structured diagnostics, exposed via a CLI. The DSL is treated as an authoring convenience; the paper's core claims are verified via IR-level bench verification and hardware audits rather than via DSL-specific novelty.

IR + MUST invariants (validation contract). The IR contract is defined as a JSON shape mirrored by `nema_ir.proto`, and the implementation includes an IR validator (`nema/ir_validate.py`) that enforces invariants before execution/compilation. The validator invariants evidenced in the current state include:

- uniqueness of node/edge identities,
- edges reference existing nodes,
- CHEMICAL edges are directed; GAP edges are symmetric,
- non-negative conductance,
- `canonicalOrderId` required,
- `license.spdx` must be in an allowed list,
- `graph.external` references must exist and (when non-placeholder) match the referenced file's SHA-256.

This validation layer is part of the bit-exactness threat model: it removes ambiguous or under-specified inputs before they can introduce divergence.

Deterministic execution and bit-exact instrumentation (software reference path). The determinism model is explicit in the spec extracts included in the input: node iteration is index-ordered and the snapshot rule is intended to make updates order-independent for a tick. The reference implementation is `nema/sim.py`. Bit-exactness is defined operationally: two executions are bit-exact if and only if all per-tick digests match; digests are computed by collecting `V` in node-index order, packing each raw voltage as signed int16 little-endian, and hashing the packed byte array with SHA-256. The numeric contract is fixed-point with RNE + saturation, and overflow behavior is SATURATE-only (no wrap).

hwtest / hardware compilation pipeline (evidence-producing path). The repo includes the pipeline components needed to compile and record evidence across stages:

- simulation/reference execution (`nema/sim.py`),
- HLS code generation (`nema/codegen/hls_gen.py`),
- a hwtest runner (`nema/hwtest.py`),
- a schema for the resulting report (`tools/bench_report_schema.js`
- and a gate script (`tools/audit_min.py`) that turns these artifacts into pass/fail criteria.

The intended evidence outputs follow repo conventions and include:

- `**/bench_report.json`,
- `golden/digest.json`,
- `golden/trace.jsonl`,
- generated HLS sources under `hls/*`,
- and toolchain reports under `hw_reports/*`.

At the paper-claim level, the hardware pipeline is treated as verified only when the hardware audit gate returns `decision=GO` (see Claim C2 verification path) and includes evidence of toolchain availability, HLS execution/report capture, and Vivado implementation timing parse (G3).

3.2 Artifact and Reproducibility

Artifact contract (what must exist / what can be regenerated). The artifact pack is organized around a small set of contracts that reviewers can re-run and/or inspect:

1. **Semantic/source contracts**
   - `spec.md` (semantic contract) and `nema_ir.proto` (schema mirror).
   - Benchmark manifests for core evaluation: `benches/B1_small/m` `benches/B3_kernel_302_7500/manifest.json`, `benches/B4_real_connectome/manifest.json`.
   - Core IR exemplars referenced by those manifests (e.g., `example_b1_small_subgraph.json`, `example_b3_kerne` and B4 external bundle IR when applicable).
2. **Generated evidence contracts**
   - A hardware build directory `build_hw/` that contains, at minimum, `**/bench_report.json` and `**/hw_reports/**/*.(rpt|xml|log)` (or whatever the parser consumes).
3. **Paper-facing artifact index and summary**
   - Tables: `papers/paperA/artifacts/tables/gates_summary.` and `papers/paperA/artifacts/tables/gates_summary.md` (software/hardware gate status summary).
   - Figures: `papers/paperA/artifacts/figures/gates_status` (caption currently marked missing in the index).
   - Evidence pointers: `papers/paperA/artifacts/evidence/aud` and `papers/paperA/artifacts/evidence/audit_hardware` referenced by the artifact index.
   - Manifest summary metadata (git HEAD, tool versions, artifact manifest SHA-256, and counts of tables/figures/evidence).

Gates (software/hardware) and what GO means. The artifact index records both software and hardware audits as `decision=GO` and highlights the key criteria that are treated as gate-defining:

- software GO includes `dslReady`, `digestMatchAll`, `benchVerifyOk`, `b3Evidence302_7500`, and `graphCountsNormalized`.
- hardware GO includes `hardwareToolchainAvailable`, `hardwareEvidenceG0b`, `hardwareEvidenceG2`, `hardwareEvidenceG3` and `digestMatchAll`. A compact snapshot of both gates is also included as `gates_status.txt`.

How to reproduce (reviewer-facing commands, relative to the repo). The paper's current claims are designed to be verified via the following scripts/commands:

Bit-exactness on core benches (Claim C1):

```
python -m nema bench verify benches/B1_small/manifest.json --outdir build/paper_verify/b1
python -m nema bench verify benches/B3_kernel_302_7500/manifest.json --outdir build/paper_verify/b3
```

PASS criterion: `ok == true` and `mismatches == []` in the verify JSON output.

Hardware evidence gate (Claim C2):

```
python tools/audit_min.py --mode hardware
```

PASS criterion: exit code `0`, `decision == GO¨`, and the relevant `criteria.*` flags are true (toolchain available, G0b/G2, and Vivado implementation evidence as tracked by the gate).

Full reproducibility gate bundle (Claim C3):

```
python -m pytest -q
python tools/audit_min.py --mode software
python tools/audit_min.py --mode hardware
# optional:
bash tools/checkpoint_full.sh
```

PASS criterion: `pytest` exit `0`, both audits exit `0` with `decision=GO¨`, and the optional checkpoint bundle exists (when used) and references reproducibility evidence.

One-shot script (recommended ergonomics, not a claim). The input proposes a reviewer-friendly wrapper script `tools/reproduce_paper_a.sh` that runs `pytest`, both audits, hardware gates, and bench verify for B1/B3/B4, writing all outputs under `build/paper_a/`. This is explicitly described as artifact ergonomics rather than a scientific claim.

Limitations explicitly carried into the artifact story. The current artifact-based no-claims include no on-board (measured) power/latency results in the artifact at this stage. Additionally, the artifact index notes a missing manifest for `benches/B5_synthetic_family/manifest.json` in the current state, which should be treated as an incompleteness in benchmark packaging rather than as a semantic limitation.

4 Evaluation

4.1 Setup and Methodology (gates)

Our evaluation is defined as a set of machine-checkable gates (exit code + JSON contract) that collectively establish (i) software correctness and determinism, and (ii) boardless hardware-toolchain evidence. The artifact pack records the gate outcomes in a compact summary table (`papers/paperA/artifacts/t` and in the full audit JSON outputs (`papers/paperA/artifacts/evidence` `papers/paperA/artifacts/evidence/audit_hardware.json`).

Primary commands (repro contract):

- `python -m pytest -q` (regression suite)
- `python tools/audit_min.py -mode software` ⇒ `decision=GO¨`
- `python tools/audit_min.py -mode hardware` ⇒ `decision=GO¨`

Benchmark-level correctness gate (bit-exactness):

- `python -m nema bench verify . . .` for manifest-driven verification.

The current artifact summary reports `decision=GO` for both software and hardware audits.

4.2 Correctness / Bit-Exact

We evaluate correctness using bit-exactness checks exposed by the repository's verification tooling. The paper's correctness metrics are strictly those produced by scripts:

- `mismatch == 0`, operationalized as `mismatches=[]` in `nema bench verify`
- `digestMatchOk == true` (or equivalent field, if exposed)
- `bench_report.ok == true`

Core correctness claim (C1). For the base benchmarks B1 and B3, the verification command `python -m nema bench verify . . .` must return `ok=true` and `mismatches=[]` (mismatch=0), establishing bit-exact agreement between the golden path and the C++ reference runner for the evaluated trace.

Audit corroboration. The software audit summary records `benchVerifyOk=True` and `digestMatchAll=True`, and the artifact index lists successful `bench_report.json` outputs for B1 and B3 (`ok=True`).

Reproducibility aspect. In addition to per-run correctness, Claim C3 states that the repo's reproducible routine includes evidence that repeated runs produce identical digests for the B1/B3 baseline (run1==run2), stored as artifact evidence (e.g., a `repro_summary.json`-style reference in the bundle).

4.3 Hardware Evidence (boardless)

We explicitly treat hardware evidence as toolchain evidence rather than physical-board measurement. The hardware pipeline is considered supported when the hardware audit gate passes:

- `python tools/audit_min.py -mode hardware` must return `decision=GO¨` and satisfy:
  - `hardwareToolchainAvailable == true`
  - `hardwareEvidenceG0b == true`
  - `hardwareEvidenceG2 == true`

- hardwareEvidenceG3 == true (Vivado implementation evidence with timing parsed; WNS non-null in the intended contract)

The hardware metrics we rely on are those recorded in bench_report.json and named in the evaluation contract:

- hardware.qor.ii
- hardware.qor.latencyCycles
- hardware.qor.utilization.{lut,ff,bram,dsp} (when present)
- hardware.vivado.wns (when implementation exists)

The artifact summary table indicates the hardware mode is ok=true with decision=GO.

4.4 Benchmark Coverage (B1/B3/B4/B2/B6 + note B5)

We report coverage using the manifest presence and the existence of a corresponding bench_report.json marked ok=True, as recorded in the artifact index.

Included in this evaluation section:

- **B1** (manifest present): benches/B1_small/manifest.json; latest report . . . /example_b1_small_subgraph/CE/2-1 (ok=True).
- **B3** (manifest present): benches/B3_kernel_302_7500/manifest.json; latest report . . . /B3_kernel_302_7500/CE/302-7500 (ok=True).
- **B4** (manifest present; external bundle workflow): benches/B4_real_connectome/manifest.json; latest report . . . /B4_celegans_external_bundle/CE/8-12 (ok=True).
- **B2** (manifest present; mid-scale diversity): benches/B2_mid/manifest.json; latest report . . . /B2_mid_64_1024/CE/64-1024 (ok=True).
- **B6** (manifest present; delay-focused benchmark): benches/B6_delay_small/manifest.json; latest report build_hw/b6/B6_delay_small/bench_report.json with target CE/3-2 (ok=True).

Note on B5. A B5-family bench_report.json exists and is marked ok=True, but the benchmark manifest is recorded as missing (manifest=NO for benches/B5_synthetic_family/manifest.json). As a result, B5 can be discussed only as a supplementary data point unless/until the manifest-based verification path is part of the artifact contract for B5.

4.5 What is NOT claimed

We explicitly scope the evaluation to determinism/bit-exactness and boardless toolchain evidence. The paper makes the following no-claims:

1. **No ML superiority claim**: no statement that NEMA (or connectome-derived structures) outperforms other ML models or achieves SOTA accuracy/sample-efficiency.
2. **No biological fidelity claim**: no statement of biological realism; the guarantee is determinism + bit-exact execution under the spec/IR contract.
3. **No on-board power/latency claim**: hardware results are not presented as measured-on-board power/latency; hardware evidence here is derived from tool outputs captured in bench_report.json and audited by gates.

5 Related Work

5.1 Determinism and Bit-Exact Reproducibility in Numerical and ML Systems

Recent software-engineering and ML-systems work shows that "same code + same seed" is often insufficient for repeatability because of hidden variance sources in kernels, reduction order, scheduling, and floating-point non-associativity [5, 19, 24, 32]. Follow-on work proposes mitigation mechanisms such as controlling hardware nondeterminism or tracking residual stochastic behavior in ostensibly deterministic settings [2, 23, 25]. A complementary numerical-analysis line focuses on reproducible summation to recover bitwise stability under ordering changes [1, 4, 6].

Positioning relative to NEMA.. Most of this bucket either measures nondeterminism or reduces it under floating-point semantics. NEMA instead defines determinism and bit-exactness as a semantic contract for a fixed-point tick model and validates that contract through reproducible artifact gates.

5.2 Operational Semantics, Numeric Contracts, and Conformance Testing

In PL and compiler-correctness literature, one strand emphasizes explicit operational semantics and mechanization (e.g., WebAssembly mechanizations and spec tooling) [22, 27, 31]. Another emphasizes semantic refinement and translation validation at IR boundaries, notably for LLVM [15]. Verified compilation lines for floating-point programs (e.g., RealCake) show how correctness arguments can be structured around constrained numeric semantics [3]. Foundational precedents include translation validation and verified-compilation traditions [14, 20, 28].

Positioning relative to NEMA.. NEMA adopts a spec-first, conformance-driven stance with executable references and regression gates, but does not claim end-to-end mechanized proof of the full toolchain.

5.3 HLS/FPGA Compilation Correctness and QoR Evidence

Hardware compilation work spans new accelerator IR/compiler infrastructures (e.g., Calyx, ScaleHLS) [16, 29, 30], formal/validated compilation paths (e.g., Vericert, verified scheduling, verified source-to-source HLS rewrites) [10, 11, 21], and QoR prediction/design-space exploration via ML surrogates [8, 9, 26].

Positioning relative to NEMA.. NEMA is complementary: it does not present a new HLS optimizer or proof-producing HLS compiler; it provides a deterministic DSL/IR

contract plus an evidence-producing software/hardware pipeline with machine-checkable outputs.

5.4 Neuromorphic and Spiking Compilation Flows

Neuromorphic toolchains increasingly emphasize portable intermediate representations and multi-backend execution, including NIR-style interoperability goals and framework-driven backend portability [7, 17, 18]. FPGA SNN surveys summarize deployment bottlenecks and implementation diversity [13].

Positioning relative to NEMA.. NEMA overlaps at the level of graph-structured neural workloads but differs in objective: the core claim is deterministic, bit-exact, artifact-auditable execution across a fixed-point tick contract rather than biological fidelity or broad simulator portability.

6 Threats to Validity and Limitations

6.1 Threats to Validity

- **T1. Scope of benchmarks is intentionally narrow** (core: B1/B3/B4; B2/B6 optional; B5 manifest missing).
  Impact: Claims may not generalize to other graphs, schedules, or workloads beyond the included manifests.
  Mitigation (or residual risk): The paper's claims are scoped to machine-checkable commands over explicit manifests (e.g., `benches/B1_small/manifest.json`, `benches/B3_kernel_302_7500/manifest.json`, `benches/B4_real_corpus/manifest.json`) and the gates (`python tools/audit_min.py ...`). Residual risk: generalization is not claimed.
- **T2. Bit-exactness definition is digest-based and only hashes per-tick `V`**.
  Impact: Two executions could differ in other internal quantities (e.g., `A`) yet still match digests, depending on what is hashed.
  Mitigation (or residual risk): The definition is explicit: "bit-exact equal when all per-tick digests match", and digest computation is specified (collect `V` in node index order, pack int16 LE, SHA-256). The claim is stated exactly in those terms, and verified via `python -m nema bench verify ...` on B1/B3. Residual risk: any state not included in the digest is out of the bit-exact contract as currently defined.
- **T3. Bit-exactness is proven only for the evaluated trace ("traza evaluada")**.
  Impact: Longer horizons or different inputs/ticks could expose divergences not covered by the verified trace.
  Mitigation (or residual risk): The claim is explicitly bounded to the evaluated trace returned by `bench verify` (PASS iff `ok=true` and `mismatches=[]`). Residual risk: the paper does not claim bit-exactness beyond what the verifier executes.
- **T4. Determinism depends on explicit ordering rules (index-ordered iteration + snapshotRule)**.
  Impact: Any implementation that violates ordering could reintroduce nondeterminism, especially if parallelization is introduced later.
  Mitigation (or residual risk): Determinism requirements are specified (`simulation node iteration is index-ordered`; `snapshot rule guarantees order-independent results for update ordering`) and a reference implementation is named (`nema/sim.py`). Divergence is detected by the bit-exact verifier (`bench verify`) on B1/B3 for the evaluated trace.
- **T5. Hardware evidence is tool-report-based (boardless), not on-board measurement**.
  Impact: QoR/timing evidence may not match physical board measurements; power/latency on silicon is not established.
  Mitigation (or residual risk): This is an explicit no-claim ("No on-board power/latency measurement claim in current artifact"). Hardware claims are limited to what `audit_min -mode hardware` machine-checks (toolchain detectable, G0b/G2 evidence present, and G3 timing parse with non-null WNS when available).
- **T6. Toolchain-version sensitivity (Vivado/Vitis HLS)**.
  Impact: Different Vivado/Vitis versions could change II/latency/utilization/timing or even fail flows, harming reproducibility on other setups.
  Mitigation (or residual risk): Toolchain versions are recorded (Vivado/v2025.2, Vitis HLS v2025.2) and the artifact contract relies on gates (hardware `audit_min` must return `decision=GO` with `hardwareToolchainAvailable==` and evidence flags). Residual risk: reproducibility across versions is not claimed.
- **T7. Report parsing / schema drift risk (QoR + Vivado timing)**.
  Impact: If tool output formats change, the parser might mis-parse or miss fields (e.g., WNS), weakening evidence.
  Mitigation (or residual risk): Hardware gate requires evidence (`hardwareEvidenceG2==true` and `hardwareEvidenceG3==` and the pipeline is contractually captured in `bench_report.json` with a schema (`tools/bench_report_schema.json`) and audited via `python tools/audit_min.py -mode hardware`.
- **T8. External bundle workflow depends on file availability + sha256 discipline**.
  Impact: If external bundle files are not packaged, moved, or hashed inconsistently, B4 may not reproduce.
  Mitigation (or residual risk): IR validation requires external references to exist and (when sha256 is not a placeholder token) to match the file digest; B4 is included specifically to exercise this workflow. Residual risk: if a placeholder is used, integrity is weaker by design; the paper should state this clearly.

- **T9. Numeric model restrictions (RNE + SATURATE; simulator path requires `Q8.8 -> Q8.8`).**
  Impact: Limits applicability to other fixed-point formats or overflow/rounding modes; may not cover broader numeric regimes.
  Mitigation (or residual risk): This is a deliberate contract: only `SATURATE` overflow and `RNE` rounding are allowed, and current simulator path requires `Q8.8 -> Q8.8`. Type/IR validation is expected to reject invalid configurations; correctness claims are made only under this contract.
- **T10. Artifact pack is minimal (few pre-generated tables/figures); most results are script-generated.**
  Impact: Reviewers may question whether enough evidence is precomputed, or whether scripts are easy to run.
  Mitigation (or residual risk): The artifact story explicitly centers on gates + stable paths (e.g., `tools/audit_min.py`, `tools/run_hw_gates.sh`, `python -m nema bench verify ...`) and includes a recommended one-shot script (`tools/reproduce_paper_a.sh`) as ergonomics (not a claim). Residual risk: ease-of-use depends on evaluator environment.

6.2 Limitations

- No ML/accuracy superiority claims (no SOTA comparisons, no accuracy/sample-efficiency statements).
- No biological fidelity claims (no claim that dynamics match real electrophysiology/plasticity).
- No on-board power/latency measurements are included in the current artifact; hardware evidence is tool-report-based.
- Numeric contract is deliberately narrow: overflow is `SATURATE` only; rounding is `RNE` only; current simulator path requires `Q8.8 -> Q8.8`.
- Bit-exactness contract is digest-based and defined over per-tick digests (digest is over packed `V` in node index order).
- B5 is not a reproducible bench manifest in the current pack (`benches/B5_synthetic_family/manifest.json` is marked MISSING), so B5 cannot be part of the core claimed evaluation.
- B4 exercises the external bundle workflow, but in the current artifact index it is represented as `B4_celegans_external_bundle/CE/8-12`, so it validates the workflow rather than claiming full-scale connectome coverage by itself.
- The paper's pre-generated artifact tables/figures are currently limited (2 tables, 1 figure, 2 evidence JSONs); additional result tables should be generated via the provided commands rather than assumed to be pre-bundled.

A Artifact Appendix

Checklist: commands are run from the repo root. Each step has a machine-checkable PASS condition.

1. **Unit/regression suite**

   ```
   python -m pytest -q
   ```

   Expected: exit code `0` (PASS).
2. **Software gate (self-audit)**

   ```
   python tools/audit_min.py --mode software
   ```

   Expected (PASS):
   - exit code `0`
   - JSON contains `decision == ̈GO¨`
   - criteria include (as documented in the artifact index):
     - `dslReady: True`
     - `digestMatchAll: True`
     - `benchVerifyOk: True`
     - `b3Evidence302_7500: True`
     - `graphCountsNormalized: True`
3. **Bit-exactness verifier for core benchmarks (golden**

   $leftrightarrow$ **C++ reference)**

   ```
   python -m nema bench verify benches/B1_small/manifest.js
   python -m nema bench verify benches/B3_kernel_302_7500/m
   ```

   Expected (PASS) for each run:
   - the verifier JSON reports:
     - `ok == true`
     - `mismatches == []` (mismatch = 0)
     - (if exposed) `digestMatchOk == true` or equivalent
4. **External bundle workflow (B4)**

   ```
   python -m nema bench verify benches/B4_real_connectome/m
   ```

   Expected (PASS):
   - verifier completes with `ok == true`
   - workflow expectation consistent with the spec contract: external referenced files must exist; if `sha256` is not a placeholder token, it must match the file digest.
5. **Hardware pipeline generation (HLS + Vivado evidence capture)**

   ```
   bash tools/run_hw_gates.sh
   ```

   Expected (PASS by artifact contract):
   - `build_hw/` contains at least one `**/bench_report.json`
   - `build_hw/` contains `**/hw_reports/**/*.(rpt|xml|log)` (or the equivalent set consumed by the report parser)
6. **Hardware gate (self-audit)**

   ```
   python tools/audit_min.py --mode hardware
   ```

   Expected (PASS):
   - exit code `0`
   - JSON contains `decision == ̈GO¨`

- criteria include (as documented in the artifact index):
  - `hardwareToolchainAvailable: True`
  - `hardwareEvidenceG0b: True`
  - `hardwareEvidenceG2: True`
  - `hardwareEvidenceG3: True`
  - `digestMatchAll: True`

7. **Paper artifact pack sanity (pre-generated table/figure + evidence)**

   ```
   cat papers/paperA/artifacts/tables/gates_summary.md
   cat papers/paperA/artifacts/figures/gates_status.txt
   ```

   Expected (PASS):
   - `gates_summary.md` shows both software and hardware with `decision = GO` (as in the artifact index preview).
   - `gates_status.txt` exists (caption noted as missing in the artifact index; this does not affect correctness gates).

8. **Optional: checkpoint/bundle regeneration (ergonomics; not a claim)**

   ```
   bash tools/checkpoint_full.sh
   ```

   Expected: produces an evidence bundle and/or updated evidence directory consistent with the repository's checkpoint conventions (used to package reproducibility evidence).

A.1 Artifact Snapshot (pre-generated assets)

The appendix directly references the current artifact files in `papers/paperA/artifacts/`.

```
Gate status snapshot
====================
software: decision=GO ok=true
hardware: decision=GO ok=true
```

Figure 1. Gate status snapshot rendered from `papers/paperA/artifacts/figures/gates_status.txt`.

Table snapshot (`gates_summary.csv`).

```
mode,ok,decision,toolchainHwAvailable,software_ok,hardware_ok,all_ok
software,true,GO,True,True,True,True
hardware,true,GO,True,True,True,True
```

Table snapshot (`gates_summary.md`).

```
| mode | ok | decision | toolchainHwAvailable | software_ok | hardware_ok | all_ok |
|---|---|---|---|---|---|---|
| software | true | GO | True | True | True | True |
| hardware | true | GO | True | True | True | True |
```

References

[1] Willow Ahrens, James Demmel, and Hong Diep Nguyen. 2020. Algorithms for Efficient Reproducible Floating Point Summation. *ACM Trans. Math. Software* (2020). https://doi.org/10.1145/3389360

[2] Berk Atil, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, et al. 2024. Non-Determinism of "Deterministic" LLM Settings. https://doi.org/10.48550/arXiv.2408.04667 arXiv:2408.04667 [cs.CL]

[3] Heiko Becker, Robert Rabe, Eva Darulova, Magnus O. Myreen, Zachary Tatlock, Ramana Kumar, Yong Kiam Tan, and Anthony Fox. 2022. Verified Compilation and Optimization of Floating-Point Programs in CakeML. In *ECOOP 2022*. https://doi.org/10.4230/LIPIcs.ECOOP.2022.1

[4] Pierre Blanchard, Nicholas J. Higham, and Théo Mary. 2020. A Class of Fast and Accurate Summation Algorithms. *SIAM Journal on Scientific Computing* (2020). https://doi.org/10.1137/19M1257780

[5] Boyuan Chen, Mingzhi Wen, Yong Shi, Dayi Lin, Gopi Krishnan Rajbahadur, and Zhen Ming Jiang. 2022. Towards Training Reproducible Deep Learning Models. In *44th IEEE/ACM International Conference on Software Engineering (ICSE 2022)*. https://doi.org/10.1145/3510003.3510163

[6] Yu-Hsin Chou et al. 2020. Deterministic Atomic Buffering. In *53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-53)*. https://doi.org/10.1109/MICRO50266.2020.00083

[7] Travis DeWolf, Pawel Jaworski, and Chris Eliasmith. 2020. Nengo and low-power AI hardware for robust, embedded neurorobotics. https://doi.org/10.48550/arXiv.2007.10227 arXiv:2007.10227 [cs.RO]

[8] Mingzhe Gao, Jieru Zhao, Zhe Lin, and Minyi Guo. 2024. Hierarchical Source-to-Post-Route QoR Prediction in High-Level Synthesis with GNNs. https://doi.org/10.48550/arXiv.2401.08696 arXiv:2401.08696 [cs.AR]

[9] Pingakshya Goswami, Benjamin Carrion Schaefer, and Dinesh Bhatia. 2023. Machine learning based fast and accurate High Level Synthesis design space exploration: From graph to synthesis. *Integration* (2023). https://doi.org/10.1016/j.vlsi.2022.09.006

[10] Yann Herklotz, James D. Pollard, Nadesh Ramanathan, and John Wickerson. 2021. Formal Verification of High-Level Synthesis. *Proceedings of the ACM on Programming Languages* (2021). https://doi.org/10.1145/3485494

[11] Yann Herklotz and John Wickerson. 2024. Hyperblock Scheduling for Verified High-Level Synthesis. *Proceedings of the ACM on Programming Languages* (2024). https://doi.org/10.1145/3656455

[12] IEEE. 2019. IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019). https://doi.org/10.1109/IEEESTD.2019.8766229

[13] Mehrzad Karamimanesh, Ebrahim Abiri, Mahyar Shahsavari, Kourosh Hassanli, André van Schaik, and Jason Eshraghian. 2025. Spiking neural networks on FPGA: A survey of methodologies and recent advancements. *Neural Networks* (2025). https://doi.org/10.1016/j.neunet.2025.107256

[14] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* (2009). https://doi.org/10.1145/1538788.1538814

[15] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. *Proceedings of the ACM on Programming Languages* (2021). https://doi.org/10.1145/3453483.3454030

[16] Rachit Nigam, Samuel Thomas, Zhijing Li, and Adrian Sampson. 2021. A Compiler Infrastructure for Accelerator Generators. In *ASPLOS '21*. https://doi.org/10.1145/3445814.3446712

[17] Jens E. Pedersen et al. 2023. Neuromorphic Intermediate Representation: A Unified Instruction Set for Interoperable Brain-Inspired Computing. https://doi.org/10.48550/arXiv.2311.14641 arXiv:2311.14641 [cs.NE]

[18] Jens E. Pedersen et al. 2024. Neuromorphic intermediate representation: A unified instruction set for interoperable brain-inspired computing. *Nature Communications* (2024). https://doi.org/10.1038/s41467-024-52259-9

[19] Hung Viet Pham, Shangshu Qian, Jiannan Wang, Thibaud Lutellier, Jonathan Rosenthal, et al. 2020. Problems and Opportunities in Training Deep Learning Software Systems: An Analysis of Variance. In

Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20). https://doi.org/10.1145/3324884.3416545

[20] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS '98).* https://doi.org/10.1007/BFb0054170

[21] Louis-Noël Pouchet, Emily Tucker, Niansong Zhang, Hongzheng Chen, Debjit Pal, Gabriel Rodríguez, and Zhiru Zhang. 2024. Formal Verification of Source-to-Source Transformations for HLS. In *FPGA '24.* https://doi.org/10.1145/3626202.3637563

[22] Xiaojia Rao, S. Radziuk, Conrad Watt, and Philippa Gardner. 2025. Progressful Interpreters for Efficient WebAssembly Mechanisation. *Proceedings of the ACM on Programming Languages* (2025). https://doi.org/10.1145/3704858

[23] Aysha Ravi et al. 2025. Improving the Reproducibility of Deep Learning Software: An Initial Investigation through a Case Study Analysis. https://doi.org/10.48550/arXiv.2505.03165 arXiv:2505.03165 [cs.SE]

[24] Shanmugavelu et al. 2024. Impacts of floating-point non-associativity on reproducibility for HPC and deep learning applications. https://doi.org/10.48550/arXiv.2408.05148 arXiv:2408.05148 [cs.DC]

[25] Megha Srivastava, Simran Arora, and Dan Boneh. 2024. Optimistic Verifiable Training by Controlling Hardware Nondeterminism. https://doi.org/10.48550/arXiv.2403.09603 arXiv:2403.09603 [cs.CR]

[26] Zi Wang and Benjamin Carrion Schafer. 2022. Learning from the Past: Efficient High-level Synthesis Design Space Exploration for FPGAs. *ACM Transactions on Design Automation of Electronic Systems* (2022). https://doi.org/10.1145/3495531

[27] Conrad Watt, Xiaojia Rao, Jean Pichon-Pharabod, Martin Bodin, and Philippa Gardner. 2021. Two Mechanisations of WebAssembly 1.0. In *Formal Methods (FM 2021).* https://doi.org/10.1007/978-3-030-90870-6_4

[28] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *PLDI '11.* https://doi.org/10.1145/1993316.1993532

[29] Hanchen Ye et al. 2022. ScaleHLS: a Scalable High-Level Synthesis Framework with Multi-level Transformations and Optimizations. In *DAC '22.* https://doi.org/10.1145/3489517.3530631

[30] Hanchen Ye, Cong Hao, Jianyi Cheng, Hyunmin Jeong, Jack Huang, Stephen Neuendorffer, and Deming Chen. 2021. ScaleHLS: A New Scalable High-Level Synthesis Framework on Multi-Level Intermediate Representation. https://doi.org/10.48550/arXiv.2107.11673 arXiv:2107.11673 [cs.AR]

[31] D. Youn et al. 2024. Bringing the WebAssembly Standard up to Speed with SpecTec. *Proceedings of the ACM on Programming Languages* (2024). https://doi.org/10.1145/3656440

[32] Donglin Zhuang, Xingyao Zhang, Shuaiwen Leon Song, and Sara Hooker. 2021. Randomness In Neural Network Training: Characterizing The Impact of Tooling. https://doi.org/10.48550/arXiv.2106.11872 arXiv:2106.11872 [cs.LG]